

Teoria dos grafos - Trabalho 3

Professor: Daniel Ratton

Alunos: Igor Valamiel Peixoto Nunes e Julia Silva da Rocha Costa

1. Descrição da tarefa

O objetivo desta terceira parte do trabalho da disciplina de Teoria dos Grafos é dar continuidade à primeira e à segunda tarefa prática do curso, adicionando à biblioteca desenvolvida anteriormente a capacidade de manipular grafos direcionados, bem como achar distâncias em grafos com pesos negativos, utilizando o algoritmo de Bellman-Ford para essa nova funcionalidade.

2. Modificações realizadas na biblioteca

Para lidar com grafos direcionados foi criado um booleano *directed*, que agora é um novo parâmetro da função *graph*. Se *directed* = 0, o grafo não é direcionado, mas se *directed* = 1, é direcionado. Por opção, essas mudanças foram implementadas somente para grafos representados por lista de adjacência. O trecho abaixo foi adicionado ao código para abranger os grafos direcionados e não direcionados. As funções não precisaram ser modificadas, apenas a estrutura do grafo.

```
if (!directed) {
    // creating edge b -> a
    node* auxB = new node;
    auxB->vertex = a;
    auxB->next = Linklist[b];
    auxB->weight = w;
    if (Linklist[b] != nullptr) Linklist[b]->back = auxB;
    Linklist[b] = auxB;

    // adding a degree to a and b
    G_list[a]++;
    G_list[b]++;
} else {
    // adding in_degree and out_degree
    G_list[a]++;
    G_in_list[b]++;
}
```

Figura 1 - alteração na função *start*

3. Desenvolvimento

A implementação do algoritmo Bellman-Ford foi feita adicionando à *struct* principal do código a função *BellmanFord*. Como saída, a função retorna, partindo de um vértice inicial *s*, a distância e a árvore de caminhos mínimos de todos os vértices do grafo para *s*, bem como detectar a presença de ciclos negativos.

3.1 Implementação

- **dist (vector<float>):** distância de cada vértice até o vértice inicial. Todas as distâncias foram inicialmente estabelecidas com infinito.
- **parent (vector<float>):** vetor que armazena os pais de cada vértice. Para facilitar, todos os pais foram inicialmente definidos como -1.
- **level (vector<float>):** vetor que armazena o nível em que cada vértice está na árvore de busca induzida. Todos os níveis foram inicialmente estabelecidos como -1.
- **continuar (bool):** booleano que monitora quando nenhuma distância for atualizada a cada iteração. Assim que não há atualização, a interação para.
- **ciclo_negativo (bool):** booleano que indica a presença de ciclo negativo. Inicialmente recebe *false*.

```

vector<vector<float>> BellmanFord(int s) {
    float inf = numeric_limits<float>::infinity();

    vector<float> dist(n + 1, inf);
    vector<float> parent(n + 1, -1);
    vector<float> level(n + 1, -1);

    dist[s] = 0;
    level[s] = 0;

    bool continuar;
    bool ciclo_negativo = false;

    // relaxamento (n - 1) vezes
    for (int i = 0; i < n - 1; i++) {
        continuar = false; // Define o início

        if (graph_type) {
            for (int v = 0; v <= n; v++) {
                if (dist[v] == inf) continue; // Garante que as iterações serão feitas apenas com distâncias finitas

                node* current = Linklist[v];
                while (current != nullptr) {
                    int u = current->vertex;
                    float w = current->weight;

                    if (u == v){
                        if (dist[v] < 0) {
                            ciclo_negativo = true;
                            continuar = false;
                            break;
                        }
                    }
                    if (dist[u] + w < dist[v]) {
                        dist[v] = dist[u] + w;
                        parent[u] = v;
                        level[u] = level[v] + 1;
                        continuar = true;
                    }
                    current = current->next;
                }
                if (ciclo_negativo) {break;}
            }
        }

        if (!continuar) {break;}
    }

    vector<float> ciclo_vec = {ciclo_negativo ? 1.0f : 0.0f};

    return {dist, parent, level, ciclo_vec};
}

```

Figura 2 - Função de implementação do algoritmo Bellman-Ford

O algoritmo garante que não sejam necessárias $n - 1$ iterações para achar caminhos mínimos menores, consequentemente reduzindo o tempo de execução. A utilização do booleano *continuar* foi essencial para a otimização do código. No Bellman-Ford tradicional, para cada vértice são realizadas $n-1$ iterações para achar a distância mínima, mas com o *continuar*, $n-1$ repetições são realizadas apenas no pior dos casos, que não acontece o tempo todo. Essa alteração resulta em uma considerável redução no tempo de execução. Ademais, a etapa de verificação de ciclos negativos evita a execução de loopings infinitos dentro do ciclo, interrompendo assim que o primeiro ciclo é encontrado.

4. Estudo de casos

4.1 Questão 1: Determine a distância dos vértices 10, 20 e 30 para o vértice 100.

Distância	Grafo 1	Grafo 2	Grafo 3	Grafo 4	Grafo 5
10 → 100	-	10,47	4,63	9,04	37,85
20 → 100	-	8,48	5,42	8,83	42,63
30 → 100	-	8,71	4,67	9,23	37,86

4.2 Questão 2: Calcule o tempo médio de execução do algoritmo, fazendo a média entre 10 rodadas.

Tempo de execução - Bellman-Ford

	Grafo 1	Grafo 2	Grafo 3	Grafo 4	Grafo 5
Tempo médio (ms)	-	126	1359	11794	93238

4.3 Questão 3: Calcule e compare, com o algoritmo de Dijkstra, a distância dos vértices 10, 20 e 30 para o vértice 100 com os valores obtidos pelo Bellman-Ford. Calcule o tempo médio de execução e compare com o Bellman-Ford.

Nessa etapa a recomendação foi inverter o grafo e executar o Dijkstra a partir do vértice 100. Essa implementação faz com que ao invés de usar o algoritmo para os vértices 10, 20 e 30, seja necessário apenas executar para o vértice 100. Essa estratégia faz com que em apenas uma execução seja possível encontrar as distâncias de 10, 20 e 30, respectivamente, para o vértice 100. Essa abordagem é bastante útil para grafos muito grandes.

```
if (negative) {cout << "Esse grafo possui pesos negativos.\n";}
else {
    vector<vector<float>> edges_2;
    for (auto i : edges) {edges_2.push_back({i[1], i[0], i[2]});}
    sort(edges_2.begin(), edges_2.end());

    graph g_2(edges_2, n, m, directed, 1, 1);
}
```

Figura 3 - Inversão do grafo

Distância dos vértices + Tempo médio de execução - Dijkstra

Distância	Grafo 1	Grafo 2	Grafo 3	Grafo 4	Grafo 5
10 → 100	-	10,47	4,63	9,04	37,85
20 → 100	-	8,48	5,42	8,83	42,63
30 → 100	-	8,71	4,67	9,23	37,86
Tempo médio (ms)	-	74,64	597,54	8789,90	85398,20

Obs: Para o grafo 1, nenhum dos testes pode ser rodado pois possui ciclos (e consequentemente pesos) negativos. Assim, nem o Dijkstra nem o Bellman-Ford podem ser executados.

5. Análise

Para os grafos analisados é notório que os tempos de execução do Bellman-Ford foram mais lentos quando comparados ao Dijkstra, o que era esperado. O algoritmo de Bellman-Ford possui uma complexidade $O(n \cdot m)$, enquanto no Dijkstra implementado com heap é $O((m + n) \cdot \log(n))$. Assim, o esperado era que Dijkstra fosse mais rápido, o que aconteceu.

Válido ressaltar que o grupo realizou uma otimização no algoritmo de Bellman-Ford, mais especificamente na parte de verificação de ciclos negativos. Antes, a verificação estava sendo realizada após a atualização das distâncias e utilizando outros dois loopings de *for*. Após a alteração essa verificação de ciclos negativos passou a ser realizada antes da atualização de distâncias e dentro do looping principal da função, o que resultou na queda de tempo.

6. Considerações finais

O grupo, durante a confecção do projeto, precisou ter diferentes ideias para as implementações requisitadas pelo professor, dentre elas destaca-se a verificação de ciclos. O ganho de desempenho após essa otimização foi gigantesco, dado que nas primeiras tentativas (especialmente para o grafo 1) saímos de uma verificação que demorava mais de 20 minutos para uma levou bem menos tempo. Também achamos interessante a versatilidade do algoritmo de Bellman-Ford para diferentes situações, se mostrando mais robusto que o Dijkstra para certas aplicações. Outro ponto interessante foi a descoberta da estratégia de inversão de grafo para uma execução mais rápida do Dijkstra, achamos muito útil.

Link para o código principal: https://github.com/igorvalamiel/graph_codes/blob/main/trabalho3/graph.cpp